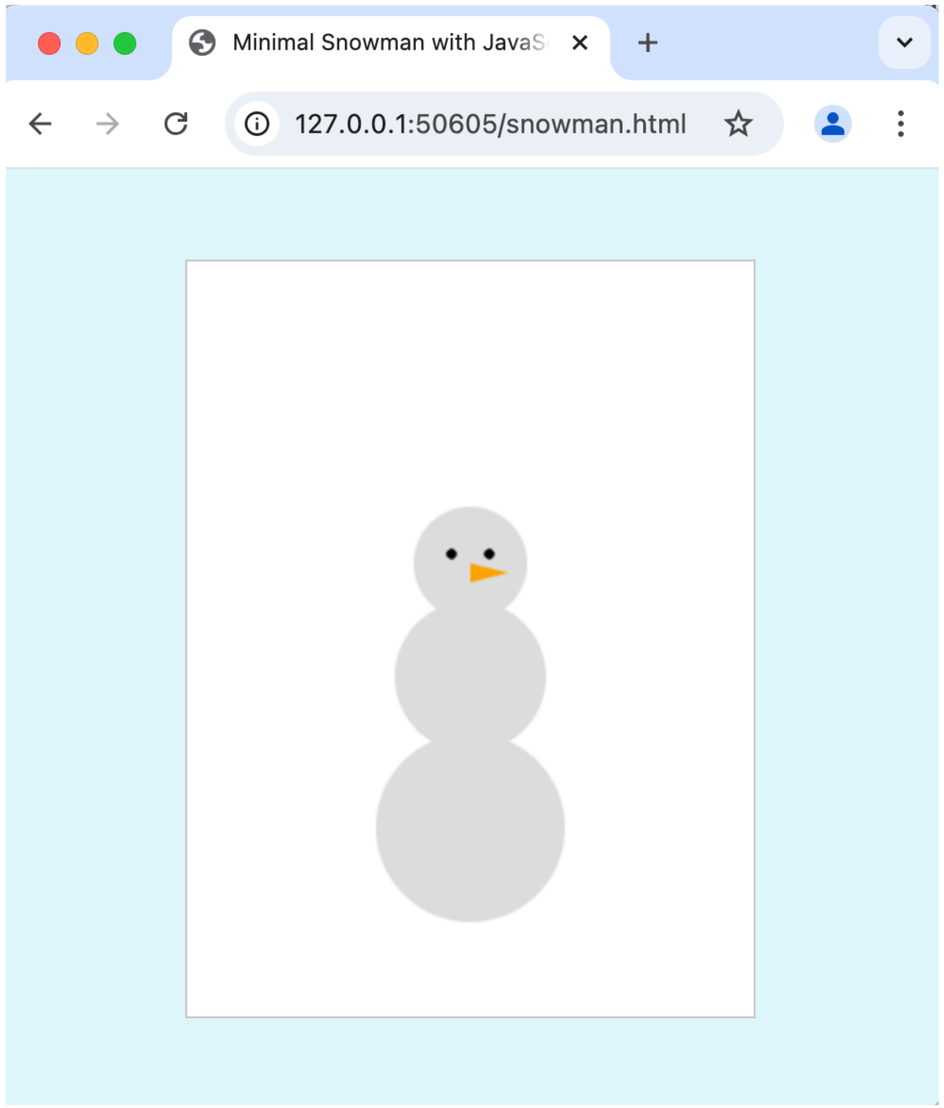
Javascript



http://localhost:8888/eclipse_projects/ajax_course_2025/examples/snowman-mvc.html

Modules (http://localhost:8888/eclipse_projects/ajax_course_2025/examples/module.html)

Introduced in es6 **each module is represented by a separate ".js" file**. We can use the **"import" or "export" statement** in a module to import or export variables, functions, classes. A module organizes a related set of JavaScript code. A module can contain variables and functions. A module is nothing more than a chunk of JavaScript code written in a file.

es6

```
// module.js
export var num = 50;
export function getName(fullName) {
   //data
};

// index.js
import {num, getName} from 'module';
console.log(num); // 50
```

Modules

In JavaScript, when you want to export functionality from a module (a file containing JavaScript code), you have two main options: **named exports** and default exports.

Named exports

```
// Module: utils.js

// Named export - Function
export function add(a, b) {
    return a + b;
}

// Named export - Variable
export const PI = 3.14;
```

Named import

```
// Module: main.js

// Importing named exports from utils.js
import { add, PI } from './utils.js';

console.log(add(5, 3)); // Output: 8
console.log(PI); // Output: 3.14
```

Modules

In JavaScript, when you want to export functionality from a module (a file containing JavaScript code), you have two main options: named exports and **default exports**.

Default exports

```
// Module: utils.js

// Default export - Function
You can export a single entity

export default
function multiply(a, b) {
    return a * b;
}
```

Default import

```
// Module: main.js

// Importing the default export from
utils.js, you can name it as you want

import multiply from './utils.js';

console.log(multiply(5, 3)); // Output:
15
```

Node.js and Npm

- Node.js is an **open-source JavaScript runtime environment** that allows you to run JavaScript code server-side outside the browser for large and complex applications

- npm stands for **Node Package Manager** and is the default package manager for Node.js. It's a vast repository of open-source software packages that can be easily installed, updated, and managed using the command line

- To initialize a project using npm, you use the **npm init** command. This command creates a **package.json** file in your project dir. The file contains metadata about the project, such as its name, version, dependencies, other configurations

node 1-test.js

```
let a = 5;
let b = 3;
let c = a * b;
console.log(`${a} * ${b} = ${c}`); \\ output 5 * 3 = 15
```

npm init - npm run start

```
\\ package.json
{
  "name": "2-server",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "start": "node server.js"
}
```

```
\\ server.js
require("http").createServer((inRequ
est, inResponse) => {
    inResponse.end("Hello from my
first Node Web server");
}).listen(8000);
```

React.js

- React.js is a popular **JavaScript library for building user interfaces**. It allows developers to create interactive and dynamic UI components for web applications

- **Component-Based Architecture**: React.js follows a component-based architecture, UIs are broken down into reusable components. Each component manages its own state

- **Virtual DOM**: React uses a virtual DOM (Document Object Model) to efficiently update the UI. Instead of directly manipulating the DOM, React creates a virtual representation of it in memory. When the state changes, React calculates the difference between the virtual DOM and the actual DOM and updates only the necessary parts, minimizing the number of DOM manipulations

- **JSX**: React uses **JSX (JavaScript XML)**, which is a syntax extension that allows you to write HTML-like code directly within JavaScript. JSX simplifies the process of building UI components by providing a familiar syntax for HTML

React.js

```
// Create a component in React.js, easier create-react-app
https://create-react-app.dev/docs/getting-started

npx create-react-app my-app
cd my-app
npm start

import React from 'react';

// Create a functional component
function App() {
  return (
    <div>
      <h1>Hello, React!</h1>
      <p>This is a simple React.js boilerplate.</p>
    </div>
  );
}

export default App
```

Typescript

- TypeScript is an **open-source programming language** developed by Microsoft.
- It is a **typed** and compiled version of JavaScript, which means it adds static types to the JavaScript language, enabling developers to write more robust and maintainable code
- TypeScript **is then translated into JavaScript** to be run in browsers or any other environment that supports JavaScript
- With advanced features like **type checking**, **inheritance**, and **interfaces**, TypeScript has become popular among developers for building large and complex applications.

Typescript

```
// Undefined and null values: javascript and typescript types…
string, number, boolean

let condition: boolean = true;
let person: string = "Bob";
const age: number = 40;
let place;

console.log("Place value: " + place + " Type: " + typeof(place));

place = "London";
console.log("Place value: " + place + " Type: " + typeof(place));

place = null;
console.log("Place value: " + place + " Type: " + typeof(place));

// [LOG]: "Place value: undefined Type: undefined"
// [LOG]: "Place value: London Type: string"
// [LOG]: "Place value: null Type: object"

Testing at https://www.typescriptlang.org/play/
```

Typescript

```
// Define a function with typed parameters

function writeValue(val: string | null) {
  console.log(`Value: ${val ?? "Fallback value"}`)
}

writeValue("London");
writeValue(null);
```

Testing at https://www.typescriptlang.org/play/

Typescript

```
// Define a function with optional parameters

function writeValue(val?: string) {
 console.log(`Value: ${val ?? "Fallback value"}`)
}

writeValue("London");
writeValue();
```

Testing at https://www.typescriptlang.org/play/

Typescript

```
// Define a function with default parameters

function writeValue(val: string = "default value") {
 console.log(`Value: ${val}`);
}

writeValue("London");
writeValue();
```

Testing at https://www.typescriptlang.org/play/

Typescript

```
// Define a function which accept rest parameters

function writeValue(val: string, ...extraInfo:
string[]) {
 console.log(`Value: ${val}, Extras: ${extraInfo}`)
}

writeValue("London", "Raining", "Cold");
writeValue("Paris", "Sunny");
writeValue("New York");

//rest parameter must be the last one and be prefixed
with …
Testing at https://www.typescriptlang.org/play/
```

Typescript

```
// Define a function which return results

function composeString(val: string) : string {
 return `Composed string: ${val}`;
}
function writeValue(val?: string) {
 console.log(composeString(val ?? "Fallback value"));
}

writeValue("London");
writeValue();
```

Testing at https://www.typescriptlang.org/play/

Typescript

```typescript
// Define a function that takes in two numbers and returns their sum
function add(x: number, y: number): number {
    return x + y;
}

// Define a variable with explicit type declaration
let greeting: string = "Hello, TypeScript!";

// Define an interface representing a person
interface Person {
    name: string;
    age: number;
}

// Create an object that conforms to the Person interface
let person: Person = {
    name: "John",
    age: 30
};

console.log(add(5, 3)); // Output: 8
console.log(greeting);  // Output: Hello, TypeScript!
console.log(person);    // Output: { name: 'John', age: 30 }
```

Typescript

```
// Define a function that takes in another
function
function getUKCapital() : string {
  return "London";
}
function writeCity(f: () => string) {
  console.log(`City: ${f()}`)
}
writeCity(getUKCapital);

// [LOG]: "City: London”
// Testing at https://www.typescriptlang.org/
play/
```

Typescript

```
// Working with arrays

let myArray = [];
myArray[0] = 100;
myArray[1] = "Adam";
myArray[2] = true;

console.log(myArray[1])

// [LOG]: “Adam"
// Testing at https://www.typescriptlang.org/
play/
```

Typescript

```
// Array with a typing annotation

let myArray: any[] = [];
myArray[0] = 100;
myArray[1] = "Adam";
myArray[2] = true;

console.log(myArray[1])

// [LOG]: “Adam”
// Testing at https://www.typescriptlang.org/
play/
```

```
// Restricting array values type

let myArray: (number | string | boolean)[] =
[];

myArray = [100, "Adam", true];

console.log(myArray)

// [LOG]: [100, "Adam", true]
// Testing at https://www.typescriptlang.org/
play/
```

Typescript

```
// Enumerating the contents of an array

let myArray: (number | string | boolean)[] = [100,
"Adam", true];

for (let i = 0; i < myArray.length; i++) {
  console.log("Index " + i + ": " + myArray[i]);
}

console.log("---");

myArray.forEach((value, index) => console.log("Index
" + index + ": " + value));

// [LOG]: "Index 0: 100"
// [LOG]: "Index 1: Adam"
// [LOG]: "Index 2: true"
// Testing at https://www.typescriptlang.org/play/
```

Typescript

```
// Using the spread operator to expand an array

let myArray: (number | string | boolean)[] = [100,
"Adam", true];

let otherArray = [...myArray, 200, "Bob", false];

otherArray.forEach((value, index) =>
console.log("Index " + index + ": " + value));

// [LOG]: "Index 0: 100"
// [LOG]: "Index 1: Adam"
// [LOG]: "Index 2: true”
// [LOG]: "Index 0: 200"
// [LOG]: "Index 1: Bob"
// [LOG]: "Index 2: false”
// Testing at https://www.typescriptlang.org/play/
```

// Built-in array methods

Method	Description
`concat(otherArray)`	This method returns a new array that concatenates the array on which it has been called with the array specified as the argument. Multiple arrays can be specified.
`join(separator)`	This method joins all the elements in the array to form a string. The argument specifies the character used to delimit the items.
`pop()`	This method removes and returns the last item in the array.
`shift()`	This method removes and returns the first element in the array.
`push(item)`	This method appends the specified item to the end of the array.
`unshift(item)`	This method inserts a new item at the start of the array.
`reverse()`	This method returns a new array that contains the items in reverse order.
`slice(start,end)`	This method returns a section of the array.
`sort()`	This method sorts the array. An optional comparison function can be used to perform custom comparisons.
`splice(index, count)`	This method removes `count` items from the array, starting at the specified `index`. The removed items are returned as the result of the method.
`unshift(item)`	This method inserts a new item at the start of the array.
`every(test)`	This method calls the `test` function for each item in the array and returns `true` if the function returns `true` for all of them and `false` otherwise.

// Built-in array methods

`some(test)`	This method returns `true` if calling the `test` function for each item in the array returns `true` at least once.
`filter(test)`	This method returns a new array containing the items for which the `test` function returns `true`.
`find(test)`	This method returns the first item in the array for which the `test` function returns `true`.
`findIndex(test)`	This method returns the index of the first item in the array for which the `test` function returns `true`.
`foreach(callback)`	This method invokes the `callback` function for each item in the array, as described in the previous section.
`includes(value)`	This method returns `true` if the array contains the specified value.
`map(callback)`	This method returns a new array containing the result of invoking the `callback` function for every item in the array.
`reduce(callback)`	This method returns the accumulated value produced by invoking the callback function for every item in the array.

Typescript

```
// Processing a data array

let myArray: (number | string | boolean)[] = [100,
"Adam", true];

let otherArray = [...myArray, 200, "Bob", false];

let sum: number = otherArray
.filter(val => typeof(val) == "number")
.reduce((total: number, val) => total + (val as
number), 0)

console.log(`Sum: ${sum}`);

// [LOG]: "Sum: 300”
// Testing at https://www.typescriptlang.org/play/
```

Typescript

```
// Obects and optional properties

let hat = { name: "Hat", price: 100 };
let boots = { name: "Boots", price: 100, category: "Snow Gear" }

function printDetails(product : { name: string, price: number,
category?: string}) {

  if (product.category != undefined) {
    console.log(`Name: ${product.name}, Price: ${product.price},
    Category: ${product.category}`);
  } else {
    console.log(`Name: ${product.name}, Price: ${product.price}
    `);
  }
}
printDetails(hat);
printDetails(boots);
// [LOG]: "Sum: 300”
// Testing at https://www.typescriptlang.org/play/
```

Typescript

```
class Animal {
    // Properties
    protected name: string;
    public age: number;

    // Constructor
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    // Method
    public introduce(): string {
        return `Hi, my name is ${this.name} and I am ${this.age} years old.`;
    }
}
```

Typescript

```typescript
// Subclass extending Animal class
class Dog extends Animal {
    // Additional property specific to Dog
    private breed: string;

    // Constructor for Dog
    constructor(name: string, age: number, breed: string) {
        // Call parent class constructor
        super(name, age);
        this.breed = breed;
    }

    // Method specific to Dog
    public bark(): string {
        return "Woof! Woof!";
    }

    // Overriding parent class method
    public introduce(): string {
        return `Hi, my name is ${this.name}, I am ${this.age} years old, and I am a ${this.breed}.`;
    }
}

// Create an instance of the Dog class
const myDog = new Dog("Buddy", 3, "Labrador");

// Accessing properties and methods of the Dog class
console.log(myDog.introduce()); // Output: Hi, my name is Buddy, I am 3 years old, and I am a Labrador.
console.log(myDog.bark());      // Output: Woof! Woof!
console.log(`My dog's age is ${myDog.age}`); // Output: My dog's age is 3

// Attempting to access protected property directly will result in an error
// console.log(`My dog's name is ${myDog.name}`);
```

Typescript

Annotazioni di Tipo

- tipizzazione statica: permette di definire i tipi per proprietà, parametri e valori di ritorno dei metodi.
- controllo in fase di compilazione: gli errori di tipo vengono individuati in fase di compilazione, riducendo il rischio di bug a runtime.

Modificatori di Accesso

- public: l'accessibilità predefinita. Le proprietà e i metodi dichiarati come public possono essere accessibili da qualsiasi parte del codice
- protected: le proprietà o metodi protected sono accessibili solo all'interno della classe stessa e dalle sue sottoclassi
- private: le proprietà o metodi dichiarati come private sono accessibili solo all'interno della classe in cui sono definiti

Vantaggi Complessivi

- maggiore robustezza: la tipizzazione e i modificatori di accesso riducono la possibilità di errori accidentali, facilitando il refactoring e la manutenzione del codice.
- chiarezza del codice: sapere esattamente quale tipo di dati ogni proprietà o parametro deve contenere rende il codice più leggibile e auto-documentante
- ereditarietà sicura: con TypeScript, l'estensione di classi e l'override dei metodi sono verificati dal compilatore, garantendo che le sottoclassi rispettino i contratti

Webpack

- Webpack is a popular module bundler for JavaScript applications. It is commonly used in modern web development to bundle and manage assets, such as JavaScript files, CSS files, images, and fonts, **into a single package** that can be efficiently served to the browser

- **Module Bundling**: Webpack treats every file in your project as a module and builds a dependency graph that maps out how these modules are interconnected. It then bundles all these modules into one or more bundles, which can be served to the browser

- **Asset Loading**: Webpack can handle various types of assets, including JavaScript, CSS, images, fonts, and more. It uses loaders to process these different types of files. For example, you can use loaders like **babel-loader** to transpile JavaScript files, **css-loader** to handle CSS files, and **file-loader** to manage image and font files

Webpack

- **Plugins**: Webpack provides a rich ecosystem of plugins that extend its functionality. Plugins can be used for various tasks such as optimizing assets, generating HTML files, extracting CSS into separate files, and more

- **Configuration**: Webpack is highly configurable through a **webpack.config.js** file, where you can specify entry points, output settings, loaders, plugins, and other options to customize the build process according to your project's requirements.

Webpack

```
// webpack.config.js

const path = require('path');

module.exports = {
  entry: './src/index.js',
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.js',
  },
  module: {
    rules: [
      {
        test: /\.js$/,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
        },
      },
    ],
  },
};
```

Building an app with Typescript, React and Webpack

Drawing canvas in the browser

Webpack

Webpack is the go-to bundler for many JavaScript projects

Setup

```
mkdir my-ts-react-webpack-app
cd my-ts-react-webpack-app
npm init -y
```

Install core dependencies

Webpack is the go-to bundler for many JavaScript projects

React and React DOM

```
npm install react react-dom
npm install --save-dev @types/react-dom
```

Typescript

```
npm install --save-dev typescript
```

TypeScript type definitions provide information about the **types of variables**, functions, and other objects in your codebase, which can help improve development workflows, catch errors early, and provide better code suggestions in editors.

Installing **@types/react-dom** specifically would provide type definitions for React's DOM-specific functionality, such as rendering components to the DOM.

A type definition file, often denoted with a **.d.ts extension**, is used in TypeScript to describe the shape of JavaScript code

tsconfig.json

Create a config file for TypeScript settings

```
tsc -init
```

```
{
  "compilerOptions": {
    "target": "es2016",
    "jsx": "react",
    "moduleResolution": "node",
    "outDir": "./dist",
  },
  "include": ["src/**/*"]
}
```

The **"moduleResolution": "node10"** configuration in a tsconfig.json file specifies the module resolution strategy TypeScript should use when resolving module names.

In this case, "node10" refers to the **Node.js module resolution algorithm** introduced in Node.js version 10.

When you import modules in TypeScript, the compiler needs to resolve these module names to **actual file paths**.

This resolution strategy searches for modules in directories specified by NODE_PATH environment variable, as well as in the **node_modules directories** of ancestor directories. It also looks for modules specified by relative or absolute paths.

tsconfig.json

The jsx flag in tsconfig.json specifies how JSX code should be transformed. Setting it to "react" means that TypeScript will convert **JSX** into **React.createElement()** calls, which is what you'd want for a typical React project.

```
{
  "compilerOptions": {
    "target": "es2016",
    "jsx": "react",
    "moduleResolution": "node",
    "outDir": "./dist",
  },
  "include": ["src/**/*"]
}
```

The **"jsx": "react"** configuration in a tsconfig.json file specifies the type of JSX transformation that TypeScript should perform when encountering JSX syntax in your code.

JSX is a syntax extension that allows you to write HTML-like code within JavaScript or TypeScript files, which is commonly used in React applications to describe UI components.

However, **browsers and JavaScript engines don't understand JSX** directly, so it needs to be transformed into regular JavaScript before it can be executed.

This means TypeScript will transform JSX syntax into **React.createElement** function calls during compilation. This is the standard way of handling JSX in React applications.

```
import React from 'react';

interface GreetingProps {
  name: string;
}

const Greeting: React.FC<GreetingProps> = ({ name }) => {
  return (
    <div>
      <h1>Ciao, {name}!</h1>
      <p>Benvenuto in questo esempio di JSX.</p>
    </div>
  );
};

export default Greeting;
```

Install webpack

```
npm install --save-dev webpack webpack-cli
webpack-dev-server
```

Create webpack.config.js in root

```
const path = require('path');

module.exports = {
  entry: ‘./src/index.tsx',
  mode: ‘development',
  output: {
    filename: 'bundle.js',
    path: path.resolve(__dirname, 'dist')
  },
  resolve: {
    extensions: ['.tsx', '.ts', '.js']
  },
  devServer: {
    open: true,
    host: "localhost",
  },
};
```

Babel setup

- Babel presets are collections of plugins that are used for transformations: ECMAScript, React JSX, Typescript

```
npm install --save-dev @babel/core babel-loader
@babel/preset-env @babel/preset-react @babel/
preset-typescript
```

Create .babelrc file in root

```
{
  "presets": [
    "@babel/preset-env",
    "@babel/preset-react",
    "@babel/preset-typescript"
  ]
}
```

This JSON snippet represents a configuration for **Babel presets**.

Babel is a popular toolchain that is mainly used **to convert ECMAScript 2015+ code (ES6+) into a backward-compatible version of JavaScript** that can be run in older browsers or environments that do not yet support the newer syntax and features of JavaScript.

Integrate Babel with webpack

- Add code in webpack.config

```
// Existing Code
module.exports = {
  // ...
  module: {
    rules: [
      {
        test: /\.(ts|tsx)$/,
        exclude: /node_modules/,
        use: 'babel-loader'
      }
    ]
  }
};
```

HTML file

- Add index.html in src

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-
width, initial-scale=1.0" />
    <title>Webpack React TS</title>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

HTML Webpack plugin

Install plugin

```
npm install --save-dev html-webpack-plugin
```

Add to webpack.config.js

```
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  // Existing Code
  plugins: [
    new HtmlWebpackPlugin({
      template: './src/index.html'
    })
  ]
};
```

Typescript React Component

Create src/index.tsx file

```
import React from 'react';
import ReactDOM from 'react-dom/client';

const App: React.FC = () => {
  return <div>Hello, world!</div>;
};
// ! not null assertion operator
const root =
ReactDOM.createRoot(document.getElementById('root')!);

root.render(<App />);
```

Add commands to package.json to run

```
"scripts": {
  "build": "webpack --mode=production --node-
env=production",
  "watch": "webpack --watch",
  "serve": "webpack serve"
},
```

Create class BoardView.tsx

```
import React from "react";

export class BoardView extends Component {
 public render() {
  return (
    <div id="controls">
      <p>Scegli il colore</p>
      <div id="buttonGreen" className="bottone"></div>
      <div id="buttonBlue" className="bottone"></div>
      <div id="buttonRed" className="bottone"></div>
      <div id="buttonYellow" className="bottone"></div>
      <div id="buttonOrange" className="bottone"></div>
      <div id="buttonBlack" className="bottone"></div>

      <div id="buttonWhite" className="bottone"></div>
      <div id="bottoneCancella" className="bottone">
        Cancella tutto
      </div>
      <br className="clearer" />
    </div>
  );
 }
}
export default BoardView;
```

Modify index.tsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import BoardView from "./components/BoardView";

const App: React.FC = () => {
  return <BoardView />;
};

// ! not null assertion operator
const root =
ReactDOM.createRoot(document.getElementById("root")!);

root.render(<App />);
```

Setup css and sass loader

```
npm install css-loader style-loader --save-dev
npm install sass-loader sass --save-dev
```

Add loader to wepack.config.js

```
module: {
  rules: [
   {
      test: /\.s[ac]ss$/i,
      use: [
        "style-loader",
        "css-loader",
        // Compiles Sass to CSS
        "sass-loader",
      ],
    },
  ],
},
```

Create ./style/style.scss and import in index.tsx

```
import "./style/style.scss";
```

```
* {
  margin:0; padding: 0;
}

canvas {
  cursor:pointer; touch-action:none;
}

.bottone {
  width: 14px; height: 14px;
  border: 1px solid #000;
  float: left;
  margin-right: 2px;
}

#controls {
  margin-left:10px;
  #buttonGreen { background-color: green;}
  #buttonBlue { background-color: blue;}
  #buttonRed { background-color: red;}
  #buttonYellow { background-color: yellow;}
  #buttonBlack { background-color: black;}
  #buttonOrange { background-color: orange;}
  #buttonWhite { background-color: white;}
  #bottoneCancella { background-color: black; color: white; padding: 1px; width: 50px; font-size: 10px; text-align:
center; height: 12px;}
}

.clearer {clear: both;}
```

Create class CanvasComponent.tsx. Resize behaviour

```
import React, { useRef, useEffect } from "react";

const CanvasComponent: React.FC<{ state: any }> = (props) => {
  const state: any = props.state;
  const canvasRef = useRef<HTMLCanvasElement>(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const handleResize = () => {
    let controlsHeight: number = document.getElementById("controls")!.offsetHeight;
    const width: number = window.innerWidth;
    const height: number = window.innerHeight - controlsHeight - 20;
    canvas.width = width;
    canvas.height = height;
    };

    handleResize(); // Initial resize
    window.addEventListener("resize", handleResize);

    return () => {
      window.removeEventListener("resize", handleResize);
    };
  }, []);

  return <canvas ref={canvasRef} />;
};

export default CanvasComponent;
```

Add State and Canvas Component to BoardView.tsx

```
import React from “react";
import { createState } from "../state";
import CanvasComponent from "./CanvasComponent";

export class BoardView extends Component {
 state = createState(this);
 public render() {
  return (
    <div id=“controls">
      <CanvasComponent state={this.state} />
      <p>Scegli il colore</p>
      <div id="buttonGreen" className="bottone"></div>
      <div id="buttonBlue" className="bottone"></div>
      <div id="buttonRed" className="bottone"></div>
      <div id="buttonYellow" className="bottone"></div>
      <div id="buttonOrange" className="bottone"></div>
      <div id="buttonBlack" className="bottone"></div>
      <div id="buttonWhite" className="bottone"></div>
      <div id="bottoneCancella" className="bottone">
        Cancella tutto
      </div>
      <br className="clearer" />
    </div>
  );
 }
}

export default BoardView;
```

Create state singleton in state.ts

```
import * as React from "react";
let stateSingleton: any = null;

export function createState(inParentComponent: React.Component): any {
  if (stateSingleton === null) {
    stateSingleton = {
      selectedColor: "#000000",
      lineWidth: 2,
      canvasToBeCleaned: false,
      changeColor: function (color: string): void {
        stateSingleton.selectedColor = color;
      },
    };
  }
  return stateSingleton;
}
```

CanvasComponent.tsx: add config properties to draw

```
import React, { useRef, useEffect } from "react";

const CanvasComponent: React.FC<{ state: any }> = (props) => {
 const state: any = props.state;
 const canvasRef: HTMLCanvasElement | null =
 useRef<HTMLCanvasElement>(null);
 let drawingToX: number = 0;
 let drawingToY: number = 0;
 let drawingFromX: number = 0;
 let drawingFromY: number = 0;
 let mouseIsDown: boolean = false;

 […]

  return <canvas ref={canvasRef} />;
};

export default CanvasComponent;
```

CanvasComponent.tsx: add event listeners

```
import React, { useRef, useEffect } from "react";

const CanvasComponent: React.FC<{ state: any }> = (props) => {
  const state: any = props.state;
  const canvasRef = useRef<HTMLCanvasElement>(null);

  […]
  function computeDrawCoordinates(e: MouseEvent): void {console.log(e.type)}

  useEffect(() => {
    const canvas: HTMLCanvasElement | null = canvasRef.current;
    if (!canvas) return;

    canvas.addEventListener("pointermove", (e: PointerEvent) => {
      computeDrawCoordinates(e);
    });
    canvas.addEventListener("pointerdown", (e) => {
      computeDrawCoordinates(e);
    });
    canvas.addEventListener("pointerup", (e) => {
      computeDrawCoordinates(e);
    });
    canvas.addEventListener("pointerout", (e) => {
      computeDrawCoordinates(e);
    });

    […]
  }, []);

  return <canvas ref={canvasRef} />;
};

export default CanvasComponent;
```

CanvasComponent.tsx: add listener callback

```
[…]
function computeDrawCoordinates(e: MouseEvent): void {
  switch (e.type) {
    case "pointermove":
      if (mouseIsDown) {
        drawingToX = e.clientX - canvasRef.current!.offsetLeft;
        drawingToY = e.clientY - canvasRef.current!.offsetTop;
        //draw();
        drawingFromX = drawingToX;
        drawingFromY = drawingToY;
      }
      break;
    case "pointerdown":
      mouseIsDown = true;
      drawingFromX = e.clientX - canvasRef.current!.offsetLeft;
      drawingFromY = e.clientY - canvasRef.current!.offsetTop;
      break;
    case "pointerup":
      mouseIsDown = false;
      break;
    case "pointerout":
      mouseIsDown = false;
      break;
  }
}
[…]
```

CanvasComponent.tsx: draw function

```
[…]
const draw = () => {
  console.log("disegna");
  const context = canvasRef.current!.getContext("2d")!;
  context.beginPath();
  context.moveTo(drawingFromX, drawingFromY);
  context.lineTo(drawingToX, drawingToY);
  context.strokeStyle = state.selectedColor;
  context.lineWidth = state.lineWidth;
  context.stroke();
  context.closePath();
};
[…]
```

Add event listener to color buttons in BoardView.tsx

```
import React from “react";
import { createState } from "../state";
import CanvasComponent from "./CanvasComponent";

export class BoardView extends Component {
  state = createState(this);
  // Function to attach click event handler to all divs with class 'bottone'
  private attachClickHandlerToButtons = () => {
    const buttons = document.querySelectorAll(".bottone");
    buttons.forEach((button) => {
      button.addEventListener("click", this.handleClick as EventListener);
    });
  };
  // Call the function to attach click event handlers when component mounts
  componentDidMount() {
    this.attachClickHandlerToButtons();
  }

  public render() {
   return (
       <div id=“controls”>
        […]
       </div>
      );
   }
}

export default BoardView;
```

Add event callback to color buttons in BoardView.tsx

```
import React from “react";
import { createState } from "../state";
import CanvasComponent from "./CanvasComponent";

export class BoardView extends Component {
 state = createState(this);
 […]
 private handleClick: EventListener = (e) => {
   const item = e.target as HTMLDivElement;
   const color = item.id.replace("button", "").toLowerCase();
   this.state.changeColor(color);
 };
 […]
 public render() {
  return (
      <div id=“controls”>
       […]
      </div>
     );
  }
}

export default BoardView;
```

Add function to state for cleaning canvas using side effects

```
import * as React from "react";
let stateSingleton: any = null;

export function createState(inParentComponent: React.Component): any {
  if (stateSingleton === null) {
    stateSingleton = {
      selectedColor: "#000000",
      lineWidth: 2,
      canvasToBeCleaned: false,
      changeColor: function (color: string): void {
        stateSingleton.selectedColor = color;
      },
      setCanvasToBeCleaned: function (toBeCleaned: Boolean): void {
        this.setState({ canvasToBeCleaned: toBeCleaned });
      }.bind(inParentComponent),
    };
  }
  return stateSingleton;
}
```

Add event listener to clean button in BoardView.tsx

```
import React from “react";
import { createState } from "../state";
import CanvasComponent from "./CanvasComponent";

export class BoardView extends Component {
 state = createState(this);
 // Function to change the state in order to use useEffect hook to clean the canvas
 private cleanBoard: MouseEventHandler<HTMLDivElement> = (e) => {
   this.state.setCanvasToBeCleaned(true);
 };

 public render() {
  return (
      <div id=“controls”>
       […]
       <div id=“bottoneCancella” className="bottone"
         onClick={this.cleanBoard}>
         Cancella tutto
       </div>
      </div>
     );
  }
}

export default BoardView;
```

CanvasComponent.tsx: add useEffect to clean canvas

```
import React, { useRef, useEffect } from "react";

const CanvasComponent: React.FC<{ state: any }> = (props) => {
 const state: any = props.state;
 const canvasRef: HTMLCanvasElement | null = useRef<HTMLCanvasElement>(null);

 […]
 useEffect(() => {
    if (state.canvasToBeCleaned) {
     const context = canvasRef.current!.getContext("2d")!;
     context.clearRect(
       0,
       0,
       canvasRef.current!.width,
       canvasRef.current!.height
     );
     state.setCanvasToBeCleaned(false);
   }
 }, [state]);
 […]

  return <canvas ref={canvasRef} />;
};

export default CanvasComponent;
```

Debugging: webpack.config.js devtools

```
module.exports = {
  entry: "./src/index.tsx",
  mode: "development",
  devtool: “source-map",
}
```

Improving code using interfaces: StateSingleton Custom Type

```
import * as React from "react";

interface StateSingleton {
  selectedColor: string;
  lineWidth: number;
  canvasToBeCleaned: boolean;
  changeColor: (color: string) => void;
  setCanvasToBeCleaned: (toBeCleaned: boolean) => void;
}

let stateSingleton: StateSingleton | null = null;

export function createState(inParentComponent: React.Component): StateSingleton {
  if (stateSingleton === null) {
    stateSingleton = {
      selectedColor: "#000000",
      lineWidth: 2,
      canvasToBeCleaned: false,
      changeColor: function (color: string): void {
        stateSingleton!.selectedColor = color;
      },
     setCanvasToBeCleaned: (toBeCleaned: boolean): void => {
      inParentComponent.setState({ canvasToBeCleaned: toBeCleaned });
    },
    };
  }
  return stateSingleton;
}
```

Components: functional vs class

Functional jsx

```
import React, { useState } from 'react';

const FunctionalComponent = () => {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <h1>Functional Component</h1>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</
button>
    </div>
  );
};

export default FunctionalComponent;
```

Class jsx

```
import React, { Component } from 'react';

class ClassComponent extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0};
  }

  increment = () => {
    this.setState((prevState) => ({
      count: prevState.count + 1,
    }));
  };

  render() {
    return (
      <div>
        <h1>Class Component</h1><p>Count: {this.state.count}
</p><button
onClick={this.increment}>Increment</button></div>
    );
  }
}

export default ClassComponent;
```

React

React component lifecycle:

- **Mount**: mounting a component involves adding or mounting the relevant node to the DOM.
- **Update**: During the update phase, the React component gets updated following the mounting process.
- **Unmount**: The final process is unmounting. The component gets deleted from the DOM at this point.

Mount

Functional

```
useEffect(() => {
  // run side-effects
}, [])
```

Class

```
componentDidMount() {
  // do something
}
```

Update

Functional

```
useEffect(() => {
  // run side-effects
}, [dependency1,
dependency2]])
```

Class

```
componentDidUpdate(prevProps)
{
  // do something
}
```

Unmount

Functional

```
useEffect(() => {
  // run side-effects

  return () => {
    // run cleanups
  }
}, [])
```

Class

```
componentWillUnMount() {
  // do cleanups
}
```